

```

❶ rw = RandomWalk()
   rw.fill_walk()

   # Plot the points in the walk.
   plt.style.use('classic')
   fig, ax = plt.subplots()
❷ ax.scatter(rw.x_values, rw.y_values, s=15)
❸ ax.set_aspect('equal')
   plt.show()

```

We begin by importing `pyplot` and `RandomWalk`. We then create a random walk and assign it to `rw` ❶, making sure to call `fill_walk()`. To visualize the walk, we feed the walk's *x*- and *y*-values to `scatter()` and choose an appropriate dot size ❷. By default, Matplotlib scales each axis independently. But that approach would stretch most walks out horizontally or vertically. Here we use the `set_aspect()` method to specify that both axes should have equal spacing between tick marks ❸.

Figure 15-9 shows the resulting plot with 5,000 points. The images in this section omit Matplotlib's viewer, but you'll continue to see it when you run *rw_visual.py*.

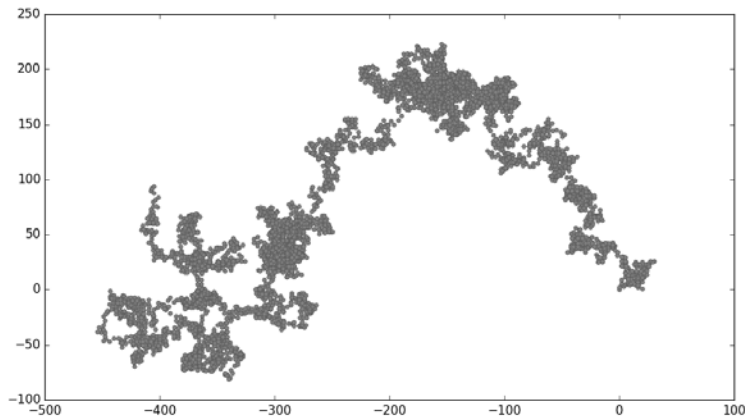


Figure 15-9: A random walk with 5,000 points

Generating Multiple Random Walks

Every random walk is different, and it's fun to explore the various patterns that can be generated. One way to use the preceding code to make multiple walks without having to run the program several times is to wrap it in a `while` loop, like this:

```

rw_visual.py import matplotlib.pyplot as plt

              from random_walk import RandomWalk

              # Keep making new walks, as long as the program is active.

```